

Career overview

Christian Rinderknecht

10 March 2016

Career

Instead of a strict chronology, I propose a thematic approach:

- ▶ Compilation (front-ends, static analysis);
- ▶ Model-based testing (telecommunications);
- ▶ Didactics of formal languages (education).

Compiler Front-ends – The case of ASN.1

The wide variety of hardware architectures and programming languages calls for the definition of a universal format for data to be exchanged over a computer network.

Abstract Syntax Notation One (ASN.1) is a standardised language (ISO, ITU-T) for defining data types whose values may be exchanged across a network between two, possibly heterogeneous peers.

ASN.1 is completed by a set of *encodings* which serialise the values into streams of bits, the two most prominent being

- ▶ the Basic Encoding Rules (BER),
- ▶ and Packed Encoding Rules (PER).

Application domains of ASN.1

- ▶ Telecommunication Management Network (SNMP, CMIP);
- ▶ Lightweight Directory Access Protocol (LDAP);
- ▶ email (X.400);
- ▶ UMTS (Radio Access Network);
- ▶ Transferred Account Procedure 3 (billing for GSM/UMTS roaming);
- ▶ 3GPP LTE (Long Term Evolution of GSM/UMTS networks);
- ▶ Multimedia conferencing systems with H.323 (for signalling: H.225, H.235, H.245, H.248, H.450.1);
- ▶ Interactive television services (MHEG);

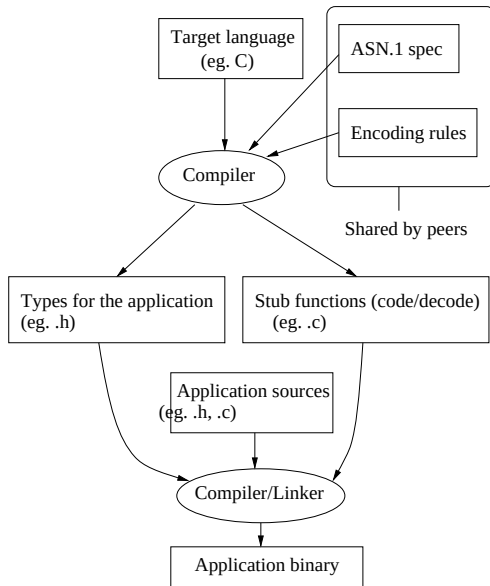
ASN.1 (usages, continued)

- ▶ Intelligent Transportation Systems (traffic signal control, electronic toll collection, emergency response etc.);
- ▶ Radio Frequency Identification (RFID);
- ▶ Aeronautical Telecommunication Network (air traffic control, flight information services, passenger communications, air-ground protocols etc.);
- ▶ Cryptography for electronic commerce over the internet (SET, PKCS 7, OpenSSL);
- ▶ Smart cards (SIM, WIM and VITALE);
- ▶ National Center for Biotechnology Information (NCBI): specification of millions of DNA sequences;

ASN.1 (usages, continued)

- ▶ client-server protocol for database access (U.S. Library of Congress: Z39.50);
- ▶ ASN.1 is embedded in other specification and programming languages like System Design Language (SDL, control), Message Sequence Charts (MSC, functional requirements), Testing and Test Control Notation (TTCN-3, tests suites), etc.
- ▶ ASN.1 is also used without formal control specification by protocol designers and tool implementors, e.g., SNMP, OpenSSL.

ASN.1 and the compilation chain



How do ASN.1 and the BER interact?

- ▶ Both peers share a common ASN.1 specification and agree upon the usage of the BER;
- ▶ each peer compile the ASN.1 specification to a set of type declarations and a set of coders/decoders (codecs) for each type;
- ▶ the target language may be different depending on the peer;
- ▶ the coders transform a value into a stream of bits according to the BER,
- ▶ the decoders transform a stream of bits into a value according to the BER (or reject it as ill-formed);
- ▶ these pieces of source code are compiled and linked separately against the corresponding communicating applications.

General issues

1. Specifications: huge (704 pages A4 10pt), complex (concepts mutually dependent).
2. Syntax: large and ambiguous grammar, dynamic extensions.
3. English cannot tell the meaning of all combinations of syntactical constructs.
4. Types denote value sets and type operators denote set operations: subtypes yield general set constraints.
5. Encoding only applies to a subset of ASN.1, defined implicitly and dependent on the encoding rules.
6. Decoding is unspecified.
7. Dynamic checks in codecs (depending on the target programming language) are unspecified.
8. Open source or free compilers: extremely rare, unmaintained, fragile, without precise error reporting.

Syntax analysis

The BNF of ASN.1 spans 23 pages, versus 19 pages for ISO C++.

When fed to Yacc as it is specified in the standards, it yields between 3,000 and 4,000 shift/reduce and reduce/reduce conflicts (for each kind!). It is ambiguous too.

The situation is actually much worse: many *tokens* are ambiguous, so if the lexer works without any extra knowledge from the parser, there are even more conflicts.

I first solved the problem of the syntax analysis on the 1990 standard, which is smaller, albeit very nasty as well.

Syntax analysis of ASN.1 1990

Too often engineers expect an off-the-shelf tool that will solve parsing problems. If Yacc is put to work,

- ▶ it may not be suited for precise error reporting (bottom-up parsing, whilst more powerful than top-down, makes it more difficult to provide syntactical context for error reporting)
- ▶ and understanding conflicts is done on the automaton, hence obscuring the relationship between the conflicts and the grammar (the parsing algorithm must be clearly held in mind at all times, because a conflict may involve several productions and a modification in one of them will change the automaton).

Real ingenuity and fortitude is needed, and I met an engineer who quit after working six months on the syntax of ASN.1 (the company gave up and is still paying an expensive license for a frontend).

Syntax analysis of ASN.1 1990 (cont.)

Nevertheless, I devised a method and tools that help. It took me three months to obtain an LL(1) grammar of ASN.1 and the corresponding recursive descent parser, which was mathematically proved correct and complete with respect to the standard BNF, and which reported syntax errors very accurately.

Beyond the academic world, it was employed by some private companies. The case I know best is that of the (ex) Southwestern Bell, where they wanted to check the validity of their ASN.1 database, used by all the subsidiaries of the group to define their telecommunication protocols.

Syntax analysis of ASN.1 1990 (cont.)

The programming language used to implement the parser was OCaml, but the open source project Cryptix used the underlying, LL(1) grammar to reimplement my parser in Java. See

<http://cryptix-asn1.sourceforge.net/ASN1-004.grammar>

Cryptix is an early library (still in use) for programming cryptography in Java. (Check the ChangeLog file and search for errors related to “parsing”.)

Incredibly short presentation of ASN.1

Primitive types

- ▶ `ok BOOLEAN ::= TRUE`
- ▶ The NULL type has only one value, also noted NULL;
`zero INTEGER ::= 0`
`DayInTheYear ::= INTEGER {first(1), last(356)}`
`newYearsEve DayInTheYear ::= last`
- ▶ `SynchroIndicator ::= ENUMERATED {serial,parallel}`
`synchro SynchroIndicator ::= serial`
- ▶ `pi REAL ::= 3.14159`
`micron REAL ::= {mantissa 1,base 10,exponent -6}`

More primitive types

- ▶ `byte BIT STRING ::= '0D'H -- or '00001110'B`
`T ::= BIT STRING {msb(7), lsb(0)}`
`v T ::= {msb, lsb} -- or '10000001'B`
- ▶ The OCTET STRING type is similar to the BIT STRING for eight-bit multiples strings.
- ▶ The OBJECT IDENTIFIER and RELATIVE-OID types aim at referencing other ASN.1 modules (a set of type and value declarations) at an international level, by means of a path in a standard tree.
- ▶ For historical reasons, there are many string types.

Some constructed types

The SET type corresponds to the record-like structures in programming languages:

```
PersonInfo ::= SET {age INTEGER, married BOOLEAN}  
i PersonInfo ::= {married TRUE, age 46}
```

Some fields can be marked as optional or having a default value:

```
Point ::= SET {x REAL DEFAULT 0, y REAL DEFAULT 0}  
origin Point ::= {}           -- or {x 0.0, y 0.0}
```


Some constructed types (cont.)

A real example:

```
DataAcknowledgementTPDU ::= SET {  
    destRef          Reference,  
    yr-tu-nr         TPDUnumber,  
    checkSum         CheckSum OPTIONAL,  
    subSeqNr         SubSequenceNumber DEFAULT 0,  
    flowControlCnf   FlowControlConfirmation OPTIONAL}
```

The SET OF type corresponds to the mathematical notion of sets with repetition:

```
T ::= SET OF INTEGER  
empty T ::= {}  
small T ::= {7, 9, 1, 1, 3}
```

Some constructed types (cont.)

The CHOICE type corresponds to a **union** in C.

```
T ::= CHOICE {x REAL, y BOOLEAN}
```

```
u T ::= x : 0.5
```

```
v T ::= y : FALSE
```

The Protocol Data Units (PDU) are CHOICE types, because they model all the possible queries and responses between two peers. A CHOICE type may be recursive, like the other constructed types. One real example (a Network Management Protocol) is:

```
CMISFilter ::= CHOICE {  
    item  FilterItem,  
    and   SET OF CMISFilter,  
    or    SET OF CMISFilter,  
    not   CMISFilter}
```

Some subtyping constraints

ASN.1 offers a very involved subtyping paradigm consisting of constraints upon recursive types, that restricts their corresponding sets of values in a set-theoretic manner, but also in a structural way.

- ▶ **Interval Constraint:** the INTEGER, REAL and (almost all) string types have totally ordered values, hence allowing interval definitions. For instance:

```
PositiveOrZeroInteger ::= INTEGER (0..MAX)
```

```
PositiveInteger ::= INTEGER (0<..MAX)
```

```
NegativeOrZeroInteger ::= INTEGER (MIN..0)
```

```
NegativeInteger ::= INTEGER (MIN.. $<0$ )
```

```
PositiveReal ::= REAL (0<..PLUS-INFINITY)
```

```
NegativeReal ::= REAL (MINUS-INFINITY.. $<0$ )
```

```
RealInterval ::= REAL (4e-5..1e-4)
```

Some subtyping constraints (cont.)

- ▶ **Value Constraint:** to restrict the set of values of a type to be a singleton:

```
Wednesday ::= Day (wednesday)
```

- ▶ **Union Constraint:** the new subtype contains the values of the first subtype and of the second subtype (keyword UNION or symbol "|"):

```
Day ::= ENUMERATED {monday, tuesday, wednesday,  
                    thursday, friday, saturday, sunday}
```

```
WeekEnd ::= Day (saturday | sunday)
```

- ▶ **Alphabet Constraint:** the strings can be restricted to be built upon a given alphabet:

```
CapitalAndSmall ::=
```

```
  IA5String (FROM ("A".."Z" | "a".."z"))
```

```
CapitalOrSmall ::=
```

```
  IA5String (FROM ("A".."Z") | FROM ("a".."z"))
```

Some subtyping constraints (cont.)

- **Size Constraint:** the values of string types may be constrained to a given sizes, introducing a constraint by the keyword `SIZE`:

```
Exactly31BitsString ::= BIT STRING (SIZE (31))
```

```
StringOf31BitsAtTheMost ::=  
    BIT STRING (SIZE (0..31))
```

```
NonEmptyString ::= BIT STRING (SIZE (1..MAX))
```

The size constraint can also apply to `SET OF` types. In that case, the semantics is very different: the values of the types are sets whose *cardinals* are specified by the size constraint:

```
SetOf5Strings ::=
```

```
    SET (SIZE(5)) OF PrintableString
```

```
SetOfStringsOf5Char ::=
```

```
    SET OF PrintableString (SIZE(5))
```

Some subtyping constraints (cont.)

- ▶ **Intersection Constraint:** the new subtype contains the values that belong to the two subtypes (keyword `INTERSECTION` or symbol `^`):

```
FrenchPhoneNumber ::=
  NumericString (FROM ("0".."9") ^ SIZE (10))
```

- ▶ **Inclusion Constraint:** to restrict a subtype to have only the values of a given subtype (optional keyword `INCLUDES`):

```
LongWeekEnd ::=
  Day ((INCLUDES (WeekEnd)) | monday)
Bis ::= Day (WeekEnd | monday)
```

Some subtyping constraints (cont.)

- ▶ **Complement Constraint:** to restrict the values of a type to *not* belong to another subtype:

```
Lipogramme ::=
  IA5String (FROM (ALL EXCEPT ("e" | "E")))
```

- ▶ **Constraint on SET OF:** to restrict the elements of a SET OF value:

```
TextBlock ::= SET OF VisibleString
AddressBlock ::=
  TextBlock (WITH COMPONENT (SIZE(1..32)))
```

Some subtyping constraints (cont.)

- ▶ Partial Constraint: to restrict *some* fields of a SET or CHOICE:

```
Quadruple ::= SET {  
    alpha ENUMERATED {in, out} OPTIONAL,  
    beta  IA5String OPTIONAL,  
    gamma SET OF INTEGER,  
    delta BOOLEAN DEFAULT TRUE}
```

we can derive a subtype whose component alpha is always present and equals in, and the component gamma always has five elements:

```
Quadruple1 ::=  
    Quadruple (WITH COMPONENTS {...,  
                                                alpha (in) PRESENT,  
                                                gamma (SIZE (5))})
```


Some subtyping constraints (cont.)

This subtype has the same values as:

```
Quadruple1 ::= SET {  
  alpha ENUMERATED {in, out} (in),  
  beta  IA5String OPTIONAL,  
  gamma SET SIZE (5) OF INTEGER,  
  delta BOOLEAN DEFAULT TRUE}
```

Practical issues in validation

ASN.1 types are considered as sets of values, thus types must have at least one (finite) value.

Because of the great expressivity of ASN.1, the compilers are not likely to fully check arbitrary combinations of subtyping constraints:

- ▶ an incorrect specification is badly rejected:

```
T ::= T produces typedef T T;  
before being rejected by a C compiler;
```

- ▶ a correction specification is rejected:

```
T ::= SET (WITH COMPONENT (0)  
          | WITH COMPONENT (1)) OF INTEGER
```

- ▶ an incorrect specification is accepted:

```
a INTEGER (0<..9) ::= 0
```

Incompleteness of the standard

- ▶ Aliasing

a T1 ::= *<a value of type T1>*

b T2 ::= a

What conditions on T1 and T2?

- ▶ Recursive types and values:

T ::= SET OF T

x T ::= {x}

y T ::= {{}}

Theoretical Problems

1. the types may have only infinite values:
`T ::= SET {a T};`
2. some value declarations may be ill-typed:
`v REAL ::= "";`
3. especially, some value references may be ill-typed, as
`a VisibleString ::= b`
`b INTEGER ::= 0;`
4. the subtype constraints may be inconsistent:
`T ::= REAL (SIZE(7));`
5. the subtypes may be empty, as
`T ::= SET ((SIZE (1)) ^ (SIZE (2))) OF REAL;`
6. the subtypes may have no value set:
`T ::= REAL (ALL EXCEPT T).`

Problems (cont.)

In other words, the issues to settle are respectively:

1. the finiteness problem;
2. the typechecking problem;
3. the type compatibility problem;
4. the constraint consistence problem;
5. the non-emptiness problem;
6. the solvability problem.

Analysis

- ▶ type compatibility $\subseteq$ typechecking;
- ▶ constraint consistence, non-emptiness $\subseteq$ solvability (the system is solved when we construct explicitly the values of each subtype);
- ▶ typechecking $\subseteq$ solvability:

$$y \text{ INTEGER } (0..9) ::= 1 \rightarrow \begin{cases} y \ A ::= 1 \\ A ::= \text{INTEGER } (0..9) \\ B ::= A \ (y) \end{cases}$$

where A and B are fresh type references.

So, finiteness and solvability are enough to get a full validation of ASN.1.

Two approaches for validation

Every ASN.1 specification is rewritten in a subset of same expressiveness, with less syntactical constructs. Two strategies:

1. Few preliminary rewrites, followed by a collection of set constraints, then fed to a generic constraint solver.
2. Lots of preliminary rewrites, simple algorithms to check solvability. (This is like taking the previous method and interleave a dedicated solver with the syntactical rewrites.)

Comparison:

1. Pros: modularity, less programming. Cons: constraints too general for off-the-shelf solvers.
2. Pros: easy checking. Cons: heavily specialised, correctness hard to assess.

We need to find a trade-off.

Set Constraints

Sets constraints are inclusions between expressions interpreted over the domain of sets of trees which may be recursively defined.

The ASN.1 types, by having a tree structure (technically, they are rational trees), fit perfectly in the usual domain of set constraints.

A set constraint has the form $\alpha \subseteq \beta$, where α and β are *set expressions*. Examples are 0 (the empty set), 1 (the set of all terms), α (a set-valued variable), $c(\alpha, \beta)$ (a constructor application), and the union, intersection or complement of set expressions. Given a set of constructors $\mathcal{C}$, where each $c \in \mathcal{C}$ has arity $a(c)$, set expressions are defined by the grammar:

$$E ::= 0 \mid 1 \mid \alpha \mid c(E_1, \dots, E_{a(c)}) \mid E_1 \cup E_2 \mid E_1 \cap E_2 \mid \neg E_1.$$

Common Application

In general, set constraints are used in the static analysis of programs: they are the smallest supersets of the sets of the computed values. Solving these constraints means constructing these approximated sets (e.g., for optimisation).

For example, consider a program with two uses of a variable v . Assume that from one use we determine that v must be a number (i.e., $v \subseteq \text{Int} \cup \text{Float}$) and from the other use that v cannot be an integer (i.e., $v \subseteq \neg \text{Int}$). Combining these constraints, we can infer that v must be a floating point number (i.e., $v \subseteq (\text{Int} \cup \text{Float}) \cap \neg \text{Int} = \text{Float}$).

Application to ASN.1

In order to apply the set constraints to ASN.1 we define specific constructors that model all the subtyping constraints (like parsing to obtain an abstract syntax tree).

Then, the problem consists in extracting set constraints from each subtype, such as their solving provide a mapping from the subtype to its set of values: this is a kind of *denotational semantics* of ASN.1.

As far as validation is concerned, if some constraints are unsolvable or the set of solutions is empty, then the specification must be rejected.

Aiken and Wimmers (1993) proposed the first solving algorithm for the class of constraints we are using, *but there are practical limitations*.

What are the Basic Encoding Rules (BER)?

- ▶ The BER are a set of rules defining the serialisation into bits of ASN.1 values;
- ▶ the bit frames (*codes* or *encodings*) have a uniform structure: a triple made of a *tag*, a *length* and some *contents*;
- ▶ the tag field is extracted from the data type,
- ▶ the contents field is extracted from the data itself,
- ▶ the length field is computed over the contents field,
- ▶ the structure allows embedding of sub-frames (in the contents field)
- ▶ and an *indefinite length* form, as a sender's option.

So, what is wrong with the BER?

Some years ago, the press reported several alleged vulnerabilities relating ASN.1 and the BER to SNMP and OpenSSL, about improper decoding (and rejection) of ill-formed BER encodings leading to

- ▶ buffer overflows,
- ▶ unspecified (non-deterministic) behaviours,
- ▶ stack corruptions,
- ▶ hence possible denials of service.

I showed that the joint design of ASN.1 and the BER is sound, and that the problems lay with the back-ends of the compilers.

Advantages of formalisation

1. For the standardisation committees:
 - ▶ Help find incompletenesses, ambiguities, inconsistencies and unneeded restrictions;
 - ▶ If an encoding is also formalised, proofs of desirable properties, like correctness, become possible.
2. For the tool makers:
 - ▶ If expressed declaratively, the formalisation can also define abstract algorithms for the front-end of an ASN.1 compiler.

Modelling (assumptions and encoding)

We start making some assumptions that abstract away low-level details and help us capturing the design principles of the BER:

- ▶ values are given in ASN.1, not in the application language (in fact they are produced in memory at run-time);
- ▶ codes are tuples, not bit series, in order to easily reason by induction on their structure;
- ▶ the network is reliable (no alteration of codes).

BER codes may not be unique for a given value, due to possible sender's options (e.g., encoding of booleans or unordered sets).

Proposition

All the BER encodings of a given value, according to a given type, are equivalent.

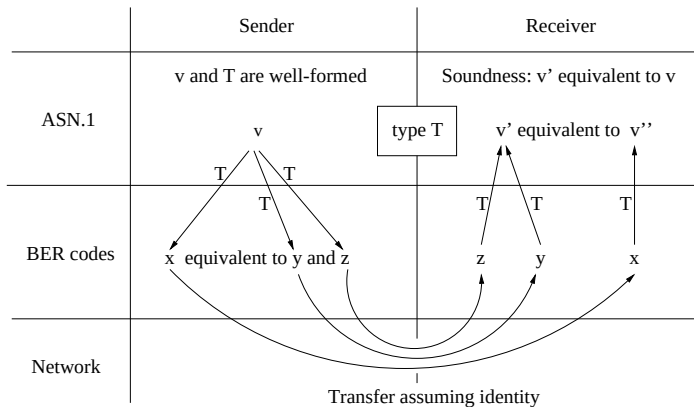
Modelling (decoding)

- ▶ The BER standard (26 pages of English) does not define the decoding at all.
- ▶ We believe that the receiver is the peer most concerned with the sound design of the BER: *the composition of encoding and decoding yields a value equivalent to the original one.* Hence we need an equivalence relationship between values (thus independent from the BER).

Theorem (Soundness)

Let v be a value of type T . Then the BER decoding of any BER encoding of v is equivalent to v .

Soundness property



Equivalence entailment

As a consequence, in order to prove the soundness property, it appears that we need to link logically the two equivalence relationships we defined separately on codes and values.

Proposition (Equivalence entailment)

Let c_1 and c_2 be two equivalent codes. Then the decoding of c_1 is equivalent to the decoding of c_2 assuming the same type.

One equivalence relies solely on the BER and the other one is inherent to ASN.1. Hence *this proposition is the link between the design of ASN.1 and the design of the BER.*

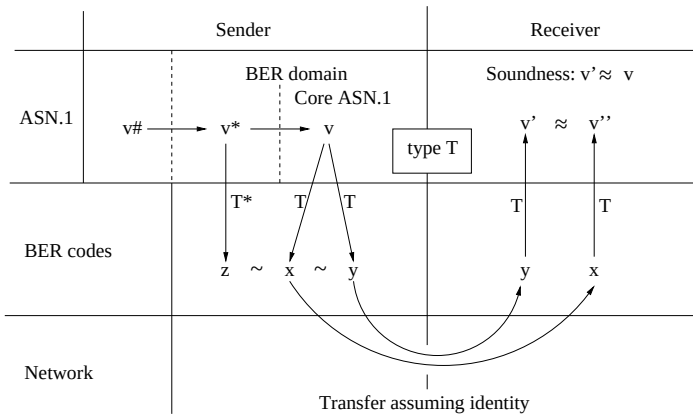
Core ASN.1

- ▶ The BER only apply to a subset ASN.1 (whose definition spans 188 pages).
- ▶ This suggests that the whole ASN.1 can be reduced to an inner subset which has the same expressivity, we call *BER domain*.
- ▶ In fact it is even possible to reduce further the BER domain to a smaller subset, we call *core ASN.1*, in which formal reasoning is easier.
- ▶ The formal definition of these reductions is achieved by means of a series of rewriting rules that preserve the set of values of the specifications (or of a given type).
- ▶ We can easily check some useful properties in core ASN.1:

Theorem (BER termination)

The encoding of core ASN.1 values with the BER always terminates.

Core ASN.1 and soundness property



Back to our model

- ▶ Let us note $\Gamma \vdash v : T \rightarrow c$ the judgement “In the environment Γ , the value v is encoded into the code c , following the type T .” (The environment models the ASN.1 module.)
- ▶ Let us note $\mathcal{D}(\Gamma, c, T)$ the decoding of c in the environment Γ according to type T .
- ▶ We already noted $v_1 \approx v_2$ the equivalence of values v_1 and v_2
- ▶ and $c_1 \sim c_2$ the equivalence of codes c_1 and c_2 .

The next step consists in defining these relations by means of systems of inference rules.

Formal model

We can now rephrase formally our propositions:

- ▶ Proposition “All the BER encodings of a given value, according to a given type, are equivalent.” becomes

Proposition

If $\Gamma \vdash v : T \rightarrow c_1$ and $\Gamma \vdash v : T \rightarrow c_2$ then $c_1 \sim c_2$.

- ▶ Proposition “The decoding of two equivalent codes lead to two equivalent values.” becomes

Proposition (Equivalence entailment)

$c_1 \sim c_2 \implies \mathcal{D}(\Gamma, c_1, T) \approx \mathcal{D}(\Gamma, c_2, T)$

- ▶ Theorem “Let v be a value of type T . Then the BER decoding of *any* BER encoding of v is equivalent to v .” becomes

Theorem (Soundness)

If $\Gamma \vdash v : T \rightarrow c$ then $\mathcal{D}(\Gamma, c, T) \approx v$.

Model-based testing

During my doctoral studies, I only considered the transmission of data. When I started as a research engineer, I got to work on the control part of communication protocols, and the generation of test suites for their implementation, derived from their specification.

This work (1998) is still quoted in research papers nowadays (this month, actually).

Didactics of programming

During my decade as a professor, I got interested in the didactics of programming, the mental models of students, the teaching methodology, and the relationship with programming language design.

I published a book entitled *Design and Analysis of Purely Functional Programs* (650 pages) and several papers in the field of didactics of informatics.

You can download my book and papers at

<http://crinderknecht.free.fr/CV/index.html>

This is also a work that is quoted nowadays.

Pet projects

- ▶ I translate poetry from Spanish to French and vice versa. I published in 1998 the only translation in French of Pablo Neruda's *Twenty Love poems and a Desperate Song*. Sales are strong!
- ▶ I occasionally write poetry myself. (Unpublished.)
- ▶ For the last ten years, I have been working on a build system for OCaml, based on GNU Make.